Flask Framework

Simone Ricci

Flask
web development,
one drop at a time

Flask

Flask is a **micro web framework** written in Python.

It is classified as a microframework because it does not require particular tools or libraries.

It has **no database abstraction layer**, form validation, or any other components where pre-existing third-party libraries provide common functions.

Flask **supports extensions** that can add application features as if they were implemented in Flask itself.

Applications that use the Flask framework include **Pinterest** and **LinkedIn**.

It was created by **Armin Ronacher** on 1 April 2010. Started as an experiment combining two existing libraries—Werkzeug (a WSGI toolkit) and Jinja2 (a template engine).

The idea was originally an April Fool's joke that was popular enough to make into a serious application: **“how small a framework could you make?”**

It has the **second-most number of stars on GitHub** among Python web-development frameworks, only slightly behind Django.

Flask vs Django

Django: **"Batteries-included" framework**. Wide range of built-in features out of the box. It includes an ORM, authentication, admin interface, form handling, and more.

Flask: **Micro-framework**. It is designed to be lightweight and flexible, includes only the essentials (routing, request handling, etc.), with additional features available through extensions.

Django: Due to its monolithic nature and built-in components, Django **may be less flexible** when it comes to customization.

Flask: The minimalistic core allows developers to choose their own libraries and tools for additional functionality.

Django: May be **slower for small apps** but well-suited for large and complex applications,.

Flask: **Faster performance for small to medium**-sized applications. Scalability can be achieved using third-party tools.

Django: Includes protection against common vulnerabilities like SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), …

Flask: **Security depends on implementation** or installed extensions.

Flask App

The first step in creating a Flask application is creating the **application object.**

```
from flask import Flask

app = Flask(__name__)
app.config.from_mapping(
    SECRET_KEY="dev",
)
app.config.from_prefixed_env()
```

Each Flask application is an instance of the **Flask** class, which collects all configuration, extensions, and views.

This is known as the "**application setup phase**", it's the code you write that's outside any view functions or other handlers.

It can be split up between different modules and sub-packages, but all code that you want to be part of your application must be imported **in order** for it to be registered.

All application **setup must be completed** before you start serving your application and handling requests.

This is because WSGI servers divide work between multiple workers.

Flask is a **WSGI application framework**. The other half of WSGI is the WSGI server.

During development, Flask, through **Werkzeug**, provides a development WSGI server with the flask run CLI command.

Blueprints

Flask uses a concept of **Blueprints** for making **application components** and supporting common patterns within an application or across applications.

Blueprints can greatly simplify how large applications work.

A Blueprint object works similarly to a Flask application object, but **it is not actually an application**.

Rather it is a blueprint of **how to construct or extend an application**.

Flask associates view functions with blueprints when dispatching requests and generating URLs from one endpoint to another.

Blueprints allow **Modularization, Reusability and Separation of Concerns.**

```python
from flask import Blueprint, render_template, abort
from jinja2 import TemplateNotFound

simple_page = Blueprint('simple_page', __name__,
                        template_folder='templates')

@simple_page.route('/', defaults={'page': 'index'})
@simple_page.route('/<page>')
def show(page):
    try:
        return render_template(f'pages/{page}.html')
    except TemplateNotFound:
        abort(404)
```

```python
from flask import Flask
from yourapplication.simple_page import simple_page

app = Flask(__name__)
app.register_blueprint(simple_page)
```

Blueprints vs Django App

Django Apps are **full-fledged components** that have a specific folder structure (models, views, templates, static files, and other Django-specific configurations)

Flask Blueprints are more **lightweight and flexible**. Blueprints are designed to be more of a logical grouping of related functionality.

Django Apps are **automatically discovered** when they are added to the INSTALLED_APPS list in settings.py.

Flask Blueprints **need to be explicitly registered** with the main Flask app using the register_blueprint() method.

Django Apps can define database models, migrations, and directly interact with the database.

Flask Blueprints **don't inherently handle database models**. If you're using something like SQLAlchemy with Flask, you'll manually define your models and connect them with your Blueprints.

Django Apps typically **define their own URL routing patterns** inside the app's urls.py file, and the routes are included in the global urls.py of the main project.

Flask Blueprints define t**heir routes within the Blueprint itself.**

Routing

Use the **route()** decorator to bind a function to a URL.

```
@app.route('/')
def index():
    return 'Index Page'

@app.route('/hello')
def hello():
    return 'Hello, World'
```

You can add variable sections to a URL by marking sections with **<variable_name>**.

The function then receives the <variable_name> as a keyword argument.

Optionally, you can use a converter to specify the type of the argument like <**converter**:variable_name>.

```
@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return f'User {escape(username)}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # show the post with the given id, the id is an integer
    return f'Post {post_id}'

@app.route('/path/<path:subpath>')
def show_subpath(subpath):
    # show the subpath after /path/
    return f'Subpath {escape(subpath)}'
```

URL Building

To build a URL to a specific function, use the **url_for()** function.

It accepts the **name of the function** as its first argument and **any number of keyword** arguments, each corresponding to a variable part of the URL rule.

Unknown variable parts are appended to the URL **as query parameters.**

```
from flask import url_for

@app.route('/')
def index():
    return 'index'

@app.route('/login')
def login():
    return 'login'

@app.route('/user/<username>')
def profile(username):
    return f'{username}\'s profile'

with app.test_request_context():
    print(url_for('index'))
    print(url_for('login'))
    print(url_for('login', next='/'))
    print(url_for('profile', username='John Doe'))
```

HTTP Methods

By default, a route only answers to **GET** requests.

The **methods** argument of the **route()** decorator allow to handle different HTTP methods.

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return do_the_login()
    else:
        return show_the_login_form()
```

You can also **separate views for different methods** into different functions.

Flask provides a shortcut for decorating such routes with **get(), post()**, etc. for each common HTTP method.

```
@app.get('/login')
def login_get():
    return show_the_login_form()

@app.post('/login')
def login_post():
    return do_the_login()
```

Rendering Templates

Flask configures the **Jinja2** template engine for rendering Template.

To render a template you can use the **render_template()** method.

You need to provide the **name of the template** and the variables you want to pass to the template engine as keyword arguments, if needed.

Flask will look for templates in the **templates** folder.

```python
from flask import render_template

@app.route('/hello/')
@app.route('/hello/<name>')
def hello(name=None):
    return render_template('hello.html', name=name)
```

```html
<!doctype html>
<title>Hello from Flask</title>
{% if name %}
  <h1>Hello {{ name }}!</h1>
{% else %}
  <h1>Hello, World!</h1>
{% endif %}
```

Inside templates you also have access to the **config**, **request**, **session** objects as well as the **url_for()**

The Request Object

The request object used by default in Flask is an instance of class **flask.Request(environ, populate_request=True, shallow=False).**

The current request method is available by using the **method** attribute.

To **access form data** (data transmitted in a POST or PUT request) you can use the **form** attribute.

To access parameters submitted in the URL (?key=value) you can use the **args** attribute.

```
@app.route('/login', methods=['POST', 'GET'])
def login():
    error = None
    if request.method == 'POST':
        if valid_login(request.form['username'],
                       request.form['password']):
            return log_the_user_in(request.form['username'])
        else:
            error = 'Invalid username/password'
    # the code below is executed if the request method
    # was GET or the credentials were invalid
    return render_template('login.html', error=error)
```

```
searchword = request.args.get('key', '')
```

Responses

The return value from a view function is automatically converted into a **response** object.

If the return value is a string it's converted into a response object with the string as response body, a 200 OK status code and a text/html mimetype.

If the return value is a **dict or list, jsonify()** is called to produce a response.

The logic that Flask applies to converting return values into response objects is as follows:

1) If a response object of the correct type is returned it's directly returned from the view.
2) If it's a **string**, a response object is created with that data and the default parameters.
3) If it's an iterator or generator returning strings or bytes, it is treated as a **streaming** response.
4) If it's a **dict or list**, a response object is created using **jsonify()**.
5) If a **tuple** is returned the items in the tuple **can provide extra information**. Such tuples have to be in the form (response, status), (response, headers). The status value will override the status code and headers can be a list or dictionary of additional header values.

APIs with JSON

It's easy to get started writing such an API with Flask.

If you return a **dict or list** from a view, it will be converted to a **JSON response**.

Returning a dict from a view will pass the data to the **jsonify()** function, which will **serialize** any supported JSON data type.

That means that all the data in the dict or list **must be JSON serializable**.

For complex types such as database models, you'll want to use a serialization library to convert the data to valid JSON types first.

```
@app.route("/me")
def me_api():
    user = get_current_user()
    return {
        "username": user.username,
        "theme": user.theme,
        "image": url_for("user_image", filename=user.image),
    }

@app.route("/users")
def users_api():
    users = get_all_users()
    return [user.to_json() for user in users]
```

SQLAlchemy ORM

SQLAlchemy is the **Python SQL toolkit** and **Object Relational Mapper** that gives application developers the full power and flexibility of SQL.

It provides a full suite of well known enterprise-level persistence patterns, designed for efficient and high-performing database access, adapted into a simple and Pythonic domain language.

```
from sqlalchemy import Integer, String
from sqlalchemy.orm import Mapped, mapped_column

class User(db.Model):
    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(unique=True)
    email: Mapped[str]
```

After all models and tables are defined, call **create_all()** to create the table schema in the database.

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

    def __repr__(self):
        return '<User %r>' % self.username
```

Flask CLI

Click is a Python package for creating beautiful command line interfaces in a composable way with as little code as necessary.

Installing Flask installs the **flask** script, a Click command line interface, in your virtualenv

The **run** command will start the development server. It replaces the **Flask.run()** method in most cases.

The flask command supports setting any option for any command with **environment variables**. The variables are named like **FLASK_APP** or **FLASK_RUN_PORT**, ...

You can use Flask's dotenv support to set environment variables automatically.

If **python-dotenv** is installed, running the flask command will set environment variables defined in the files **.env** and **.flaskenv.**

Custom flask commands are implemented using Click.

```
@app.cli.command("create-user")
@click.argument("name")
def create_user(name):
    ...
```